

DA-FDIDS

代码流程图

B0-B8 消融变体 × 域内 / 跨域 / 概念漂移 三类协议

基于 main.py DAFDIDSRunner 流水线 | pretrain_epochs=10 meta_epochs=100 repetitions=10

2026 年 6 月

实验矩阵总览

变体	描述	基础编码	LoRA	缓存	GRL	MMD	稳定 LoRA	RBF+	自适应 α
B0	DIDS-MFL 基线	—	—	—	—	—	—	—	—
B1	+ 基础编码器	✓	—	—	—	—	—	—	—
B2	+LoRA 适配器	✓	✓	—	—	—	—	—	—
B3	F-DIDS-MFL (+ 缓存)	✓	✓	✓	—	—	—	—	—
B4	+GRL 域对抗	✓	✓	✓	✓	—	—	—	—
B5	+MMD 分布对齐	✓	✓	✓	—	✓	—	—	—
B6	+ 稳定 LoRA	✓	✓	✓	—	—	✓	—	—
B7	+RBF 缓存 +MHA	✓	✓	✓	—	—	—	✓	—
B8	完整 DA-FDIDS	✓	✓	✓	✓	✓	✓	✓	✓

三类协议：域内（同数据集训练 + 测试） | 跨域（源数据集→目标数据集） | 概念漂移（同数据集时间切分）

Alpha 扫描策略：B0-B2 固定 $\alpha=0.7$ | B3-B8 遍历 $\alpha \in \{0.0, 0.3, 0.5, 0.7, 1.0\}$ 共 5 个值

融合公式： $S_{\text{final}} = \alpha \cdot S_{\text{MFL}} + (1-\alpha) \cdot S_{\text{cache}}$ （ $\alpha=0$ 纯缓存检索 / $\alpha=1$ 纯 MFL 相似度）

总体架构: DAFDIDSRunner.run() 主流程

加载数据
raw_train / raw_test
(.pt 文件)

数据过滤
filtered_data()
→ meta pools

初始化模型
TrafficEncoder/TGNMemory
MGD/SelfExpr/MFL

参数解析
parse_args()
构建 CLI 参数

for epoch in range(pretrain_epochs): pre_train() → 时序图记忆 + 多图扩散 + 二分类 / 多分类预测器训练

maybe_load_checkpoint()?
(流水线中已禁用)

```
for i in range(repetitions):  
    meta_train() → run_episode('train') × 100 轮 # 默认 10 次重复  
    meta_val() → run_episode('val') × 1 轮 # 元训练: 每轮采样支撑集 + 查询集  
    meta_test() → run_episode('test') × 1 轮 # 元验证  
    summarize(results) → RUN_METRICS_JSON # 元测试 → 收集 F1/NMI 指标  
                                         # 汇总均值 ± 标准差
```

三类协议仅数据集参数不同: 域内 (train=test) / 跨域 (train≠test, 同维度) / 漂移 (train=test, 时间切分)

B0: DIDS-MFL 基线

纯 DIDS-MFL 流水线，不加任何新组件

激活参数：（无额外参数 - 纯 DIDS-MFL 基线）

Alpha: 0.7 固定

加载数据
(raw .pt, 过滤)

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

SelfExpr(cat(支撑, 查询)) $\rightarrow$ c, x2
MFL(ADMM 优化 C) $\rightarrow$ 增强相似度 L1

enhance_sim_matrix(C1)
 $\rightarrow$ 最终相似度矩阵 L

class_scores_from_similarity() $\rightarrow$ softmax $\rightarrow$ 预测类别 $\rightarrow$ F1/NMI 评估

B1: + 基础编码器

在 TGNMemory 之前注入 TrafficEncoder

激活参数: `--use_foundation_encoder`

Alpha: 0.7 固定

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

SelfExpr(cat(支撑, 查询)) → c, x2
MFL(ADMM 优化 C) → 增强相似度 L1

enhance_sim_matrix(C1)
→ 最终相似度矩阵 L

class_scores_from_similarity() → softmax → 预测类别 → F1/NMI 评估

B2: +LoRA 在线适配

Foundation 编码器 + LoRA 少样本在线适配

激活参数: `--use_foundation_encoder --use_lora_adapt`

Alpha: 0.7 固定

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

`online_lora_adapt()`
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

`edge_vectors()`
`z=catsrc,dst`

`SelfExpr(cats 支撑, 查询)) → c, x2`
MFL(ADMM 优化 C) → 增强相似度 L1

`enhance_sim_matrix(C1)`
→ 最终相似度矩阵 L

`class_scores_from_similarity()` → softmax → 预测类别 → F1/NMI 评估

B3: F-DIDS-MFL (+ 缓存融合)

加入缓存融合相似度检索, α 参数扫描

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg $\rightarrow$ 64 维嵌入

online_lora_adapt()
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

SelfExpr(cat(支撑, 查询)) $\rightarrow$ c, x2
MEI (ADMM 优化 C) $\rightarrow$ 增强相似度 I 1

cache_similarity_matrix()
S_cache = cos/RBF(支撑, 查询)

enhance_sim_matrix(C1)
 $\rightarrow$ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

class_scores_from_similarity() $\rightarrow$ softmax $\rightarrow$ 预测类别 $\rightarrow$ F1/NMI 评估

B4: +GRL 域对抗

B3 基础上加入梯度反转层域对抗预训练

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion --use_grl_da`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

online_lora_adapt()
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

GRL 损失
(域对抗)

SelfExpr(cat(支撑, 查询)) → c, x2
MFL (ADMM 优化 C) → 增强相似度 I 1

cache_similarity_matrix()
S_cache = cos/RBF(支撑, 查询)

enhance_sim_matrix(C1)
→ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

class_scores_from_similarity() → softmax → 预测类别 → F1/NMI 评估

B5: +MMD 分布对齐

B3 基础上加入 MMD 支持集 - 查询集分布对齐

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion --use_mmd_align`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

online_lora_adapt()
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

MMD 损失
(分布对齐)

SelfExpr(cat(支撑, 查询)) → c, x2
MFL (ADMM 优化 C) → 增强相似度 I 1

cache_similarity_matrix()
S_cache = cos/RBF(支撑, 查询)

enhance_sim_matrix(C1)
→ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

class_scores_from_similarity() → softmax → 预测类别 → F1/NMI 评估

B6: + 稳定 LoRA 约束

B3 基础上约束 LoRA 适配前后的特征漂移

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion --use_stable_lora`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

`online_lora_adapt()`
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

`edge_vectors()`
`z=cat(src,dst)`

稳定 LoRA
MMD(z 前, z 后)

`SelfExpr(cat(支撑, 查询)) → c, x2`
`MFL (ADMM 优化 C) → 增强相似度 I 1`

`cache_similarity_matrix()`
`S_cache = cos/RBF(支撑, 查询)`

`enhance_sim_matrix(C1)`
→ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

`class_scores_from_similarity()` → softmax → 预测类别 → F1/NMI 评估

B7: +RBF 缓存 +MHA 加权

B3 基础上使用多尺度 RBF 核 +MHA 特征加权

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion --use_rbf_cache --use_mha_weighting`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg → 64 维嵌入

`online_lora_adapt()`
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

`edge_vectors()`
`z=cat(src,dst)`

RBF 核
多尺度 σ
MHA 加权

`SelfExpr(cat(支撑, 查询)) → c, x2`
`MFL (ADMM 优化 C) → 增强相似度 I 1`

`cache_similarity_matrix()`
`S_cache = cos/RBF(支撑, 查询)`

`enhance_sim_matrix(C1)`
→ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

`class_scores_from_similarity()` → softmax → 预测类别 → F1/NMI 评估

B8: 完整 DA-FDIDS

全部组件启用, 自适应缓存融合权重

激活参数: `--use_foundation_encoder --use_lora_adapt --use_cache_fusion --use_grl_da --use_mmd_align --use_stable_lora --use_rbf_cache --use_mha_weighting --adaptive_cache_alpha`

Alpha: 0.0-1.0 扫描

加载数据
(raw .pt, 过滤)

TrafficEncoder
msg $\rightarrow$ 64 维嵌入

online_lora_adapt()
3 步内循环
lr=1e-3 rank=4
仅更新 ~5K 参数

TGNMemory
+ 邻居加载器

MGD
多图扩散聚合

edge_vectors()
z=cat(src,dst)

SelfExpr(cat(支撑, 查询)) $\rightarrow$ c, x2
MFL (ADMM 优化 C) $\rightarrow$ 增强相似度 I 1

cache_similarity_matrix()
S_cache = cos/RBF(支撑, 查询)

enhance_sim_matrix(C1)
 $\rightarrow$ 最终相似度矩阵 I

融合: $S_{final} = \alpha \cdot S_{MFL} + (1-\alpha) \cdot S_{cache}$

class_scores_from_similarity() $\rightarrow$ softmax $\rightarrow$ 预测类别 $\rightarrow$ F1/NMI 评估

域内协议详解

同一数据集上随机划分训练 / 验证 / 测试集。验证各组件在已知分布上的基线表现。

训练数据：
CIC-ToN-IoT

测试数据：
CIC-ToN-IoT

→ 相同流水线：预训练 → 元训练 (100 轮) → 元验证 → 元测试 (10 次重复) → 汇总指标

核心差异：

`dataset_train = CIC-ToN-IoT` | `dataset_test = CIC-ToN-IoT`

- `split_train_val_test(data, classes, test=0.2, train=0.6)` — 按类别分层随机划分
 - 训练集和测试集来自相同的域分布 → 衡量各组件在已知环境下的增益
 - 每个变体运行 `alpha` 扫描 (B0-B2 固定 $\alpha=0.7$ / B3-B8 遍历 5 个 α 值)
 - 预期：B3 缓存融合带来最大单次跃升，B7/B8 进一步优化

跨域协议详解

在源数据集训练，在目标数据集测试。评估模型的域泛化能力。DA 模块 (GRL/MMD) 专为此场景设计。

训练数据：
CIC-ToN-IoT

测试数据：
CIC-BoT-IoT (76 维)

→ 相同流水线：预训练 → 元训练 (100 轮) → 元验证 → 元测试 (10 次重复) → 汇总指标

核心差异：

`dataset_train = CIC-ToN-IoT` | `dataset_test = CIC-BoT-IoT (76 维)`

- `raw_train = torch.load('CIC-ToN-IoT.pt')` vs `raw_test = torch.load('CIC-BoT-IoT.pt')`
 - `resolve_data_path()` 自动匹配 .pt 文件路径 (含 _new 和 CiC 前缀兼容)
 - 源域和目标域必须具有相同的特征维度 (如 76 维)，否则需特征对齐
- 预期：B0 跨域 F1 极低 (0.12)，DA 模块 (GRL/MMD/RBF+MHA) 逐步提升至 0.37
 - 跨域是核心挑战：域内 F1 可达 0.69，跨域仅 0.37 — 差距显著

概念漂移协议详解

同一数据集按时间切分训练 / 测试集，模拟网络流量随时间变化的概念漂移场景。检验缓存融合的抗漂移能力。

训练数据：
CIC-ToN-IoT

测试数据：
CIC-ToN-IoT (时间切分)

→ 相同流水线：预训练 → 元训练 (100 轮) → 元验证 → 元测试 (10 次重复) → 汇总指标

核心差异：

`dataset_train = CIC-ToN-IoT` | `dataset_test = CIC-ToN-IoT ( 时间切分 )`

- 使用同一 .pt 文件，通过时间戳切分模拟概念漂移
- 缓存融合 (B3+) 提供 training-free 即时适应能力，无需重新训练即可应对漂移
 - RBF 多尺度核 (B7) 对漂移最有效：F1 从 B0 的 0.08 跃升至 0.68
 - 预期：纯缓存检索 ($\alpha=0$) 在漂移场景下显著优于纯 MFL 学习 ($\alpha=1$)

深挖： LoRA 在线少样本适配 (B2+)

online_lora_adapt(support) 执行流程：

- ① 保存当前 LoRA 权重 → lora_state_dict()
- ② 获取可训练参数 → lora_parameters()
仅 LoRA_A ($4 \times \text{in}$) + LoRA_B ($\text{out} \times 4$)
约 5K 参数, 占总参数 <1%
- ③ for step in range(lora_steps=3):
z = encoder(sup.msg, normalize=True)
loss = proto_CE(z, labels) // 原型交叉熵
loss.backward() → opt.step()
- ④ 记录耗时和参数量 → cost_records
- ⑤ 返回 lora_state 供后续恢复

核心优势：基础编码器权重完全冻结
仅 LoRA 低秩矩阵参与梯度更新

LoRA 架构细节：

TrafficEncoder 内部线性层：
Linear(in→hid): $W_0 + B \cdot A$
Linear(hid→64): $W_0 + B \cdot A$
rank=4, $\alpha=1.0$

每轮结束后恢复：
restore_lora(state)
→ 重置到适配前状态
→ 每轮独立适配, 防止过拟合

效率对比：
全量微调：~500K 参数, ~1.5 秒
LoRA 适配：~5K 参数, ~0.12 秒
加速 12 倍, 参数减少 99%

深挖：缓存融合相似度检索 (B3+)

cache_similarity_matrix() 计算流程：

① $s = \text{encoder}(\text{support.msg}, \text{normalize}=\text{True})$
 $q = \text{encoder}(\text{query.msg}, \text{normalize}=\text{True})$
使用冻结的编码器（无梯度）

② 若启用 MHA 加权 (B7+):
 $s = \text{MHA}(s)$, $q = \text{MHA}(q)$
自适应调整特征维度重要性

③ 若启用 RBF 核 (B7+):
 $\text{sim} = \sum_i \exp(-||s-q||^2 / 2\sigma_i^2)$
 $\sigma \in \{0.1, 0.5, 1, 5, 10\}$
多尺度捕捉细 / 粗粒度相似性

④ 若启用时间衰减：
 $\text{sim} *= \exp(-|t_q - t_s| / \tau)$

融合公式与 α 自适应：

$$S_{\text{final}} = \alpha \cdot S_{\text{MFL}} + (1-\alpha) \cdot S_{\text{cache}}$$

$\alpha=0.0$: 纯缓存检索 (training-free)
对新攻击 / 概念漂移即时响应
 $\alpha=1.0$: 纯 MFL 相似度学习
需要元训练, 适应已知模式

自适应 α (B8):
 $\alpha = \text{clip}(\text{cache_}\alpha - \text{MMD}, 0, 1)$
域差异大 $\rightarrow$ 降低 $\alpha \rightarrow$ 信任缓存更多
域差异小 $\rightarrow$ 提高 $\alpha \rightarrow$ 信任 MFL 更多

关键优势：缓存写入 $<1\text{ms}$
无需任何训练, 即写即用

深挖： GRL + MMD 域自适应机制 (B4, B5, B8)

GRL (梯度反转层) 域对抗训练 :

- ① s_enc = encoder(sup.msg, norm=True)
- ② q_enc = encoder(qry.msg, norm=True)
- ③ x = cat(s_enc, q_enc) // 拼接
- ④ domain_logits = D(x)
域判别器 : 64→32→1 (三层 MLP)
- ⑤ targets: 支撑 =0(源域), 查询 =1(目标域)
- ⑥ L_grl = BCE(logits, targets)

关键机制 : 梯度反转
 $\partial L / \partial x = -\lambda \cdot \partial L_{adv} / \partial x$
编码器被迫隐藏域信息
使判别器无法区分源域 / 目标域

MMD (最大均值差异) 分布对齐 :

$MMD^2(P, Q) = ||\mu_P - \mu_Q||^2_H$
在再生核希尔伯特空间中度量
两个分布的距离

多尺度 RBF 核 :
 $K(x, y) = \sum_{\sigma} \sigma \exp(-||x-y||^2 / 2\sigma^2)$
 $\sigma \in \{0.1, 0.5, 1, 5, 10\}$
同时捕捉细粒度和粗粒度差异

$L_{mmd} = MMD^2(s_enc, q_enc)$
显式最小化支撑集和查询集的
特征分布距离

GRL + MMD: 互补机制 !
对抗方式 + 统计方式双重对齐

稳定 LoRA 约束 (B6):

$z_pre = \text{encoder}(\text{sup.msg}) \leftarrow$ LoRA 适配前的编码器输出
 $z_post = \text{encoder}(\text{sup.msg}) \leftarrow$ LoRA 适配后的编码器输出 (经过 3 步内循环更新 LoRA 权重)
 $L_stable = MMD(z_pre, z_post) \rightarrow$ 约束 LoRA 适配不偏离太远, 防止灾难性遗忘
通过 --stable_lora_weight 控制约束强度, 默认 0.1

组件依赖关系与约束规则



依赖规则（在 `main.py __init__` 中验证）：

- `--use_lora_adapt` 必须同时启用 `--use_foundation_encoder`
- `--use_grl_da` / `--use_mmd_align` / `--use_stable_lora` / `--use_mha_weighting` / `--adaptive_cache_alpha` 必须同时启用 `--use_foundation_encoder`
- `--use_rbf_cache` / `--use_mha_weighting` / `--adaptive_cache_alpha` 必须同时启用 `--use_cache_fusion`

递进路线：**B0(基线)** → **B1(编码器)** → **B2(LoRA)** → **B3(缓存 = F-DIDS-MFL)** → **B4/B5/B6/B7(独立 DA 插件)** → **B8(全部组合 = DA-FDIDS)**

核心函数： run_episode() 完整 17 步流程

- ① split_train_val_test(pool) → 按类别分层随机划分训练 / 验证 / 测试集 (60/20/20)
- ② sample_episode(split_data) → 每类采样 k_shot=5 个支撑样本 + q_query=15 个查询样本
- ③ [B6] z_pre = encoder(sup.msg) 保存 LoRA 适配前的编码器输出, 用于稳定约束
- ④ online_lora_adapt(support) → [B2+] 3 步内循环更新 LoRA 权重 (~5K 参数, 0.12 秒)
- ⑤ reset_episode_state() → 清空 TGN 记忆状态 + 邻居加载器状态
- ⑥ edge_vectors(support) → TGNMemory+MGD → cat(z_src, z_dst) = 支撑集边向量
- ⑦ edge_vectors(query) → TGNMemory+MGD → cat(z_src, z_dst) = 查询集边向量
- ⑧ [B4] GRL 损失: domain_discriminator(enc(sup), enc(qry)) → L_grl
- ⑨ [B5] MMD 损失: mmd_loss(enc(sup), enc(qry)) → L_mmd
- ⑩ [B6] 稳定 LoRA: mmd_loss(z_pre, z_post) → L_stable
- ⑪ concatenated = cat(sup_vec, qry_vec) → SelfExpr → c(系数矩阵), x2(自表达特征)
- ⑫ MFL(x2, C) → ADMM 优化 80 轮 → L1 → enhance_sim_matrix(C1) → 相似度矩阵 L
- ⑬ [B3+] cache_similarity_matrix() → S_final = $\alpha \cdot L + (1-\alpha) \cdot S_cache$
- ⑭ class_scores_from_similarity(S_final) → 每类平均相似度 → min-max 归一化 → softmax
- ⑮ 总损失 = CE(scores, labels) + ||x2-concat|| + $\lambda_grl \cdot L_grl + \lambda_mmd \cdot L_mmd + \lambda_stable \cdot L_stable$
- ⑯ loss.backward() → optimizer1.step() 更新 SelfExpr 和 DA 模块参数
- ⑰ restore_lora(state) → 恢复 LoRA 权重到适配前状态, 为下一轮准备

三类协议数据流对比

域内 In-Domain

训练 : CIC-ToN-IoT.pt
测试 : CIC-ToN-IoT.pt

相同流水线 :
预训练 → 元训练 → 元测试

相同变体 :
B0-B8 标志 /alpha 扫描

split_train_val_test()
随机 60/20/20 划分
同一分布 → 验证组件基线表现
预期最佳 $F1 \approx 0.69$ (B8)

跨域 Cross-Domain

训练 : CIC-ToN-IoT.pt
测试 : CIC-BoT-IoT.pt

相同流水线 :
预训练 → 元训练 → 元测试

相同变体 :
B0-B8 标志 /alpha 扫描

不同 .pt 文件加载
必须特征维度一致 (76d)
DA 模块 (GRL/MMD) 关键
预期最佳 $F1 \approx 0.37$ (B8)
跨域是核心挑战

概念漂移 Drift

训练 : CIC-ToN-IoT.pt
测试 : CIC-ToN-IoT.pt (时间切分)

相同流水线 :
预训练 → 元训练 → 元测试

相同变体 :
B0-B8 标志 /alpha 扫描

同一数据集时间切分
缓存 (B3+) 抗漂移关键
RBF 核 (B7) 对漂移最有效
预期最佳 $F1 \approx 0.68$ (B7)
纯缓存 ($\alpha=0$) 优于纯 MFL

pre_train() 预训练详细流程

在元训练之前运行。使用全部数据训练 TGNMemory + MGD + 分类预测器，共 pretrain_epochs 轮（默认 10 轮）。

```
for epoch in range(pretrain_epochs):
    for batch in TemporalDataLoader(pool, batch_size=50):
        batch.to(device)
        optimizer.zero_grad()

        src,dst,t,msg,label,attack = batch
        n_id = unique(cat(src,dst))
        n_id,_ = neighbor_loader(n_id)
        assoc[n_id] = arange
        z,_ = memory(n_id)
        ed,_ = nodeMap(stack(src,dst))
        z = mgd(z, ed, norm_factor)

        bin_out = bin_predictor(z[assoc[src]], z[assoc[dst]])
        mul_out = mul_predictor(z[assoc[src]], z[assoc[dst]])
        loss = criterion(bin_out, mul_out, label, attack, z)

        encoded = encode_msg(msg)
        memory.update_state(src, dst, t, encoded)
        neighbor_loader.insert(src, dst)
        loss.backward() → optimizer.step()
        memory.detach()

# 默认 : 10 轮
# CIC-ToN-IoT 约 2292 个批次
# 移至 GPU
# Adam 优化器 (lr=1e-4)

# 解包时序边数据
# 活跃节点集合
# 邻居采样 (size=20)
# 节点→索引映射
# TGNMemory 前向传播
# 边索引矩阵
# MGD 多图扩散聚合

# 二分类 ( 良性 / 恶意 )
# 多分类 (5 种攻击类型)
# 联合损失函数

# [B1+]: TrafficEncoder 编码
# 更新 TGN 节点记忆
# 更新图结构
# 反向传播 + 参数更新
# 截断梯度, 节省显存
```

预训练优化器 : Adam(lr=1e-4) 包含参数 : TGNMemory + MGD + [B1+] TrafficEncoder 中 requires_grad=True 的参数

SelfExpr 自表达学习 + MFL 多因子少样本学习

SelfExpr (自表达层):

输入 : concat(支撑向量 , 查询向量)
形状 : (way*(k+q), 2*embed_dim)
= (100, 128) 当 way=5,k=5,q=15

前向传播 :
 $c = \text{weight} \cdot \text{input}$
可学习系数矩阵 C
 $x2 = c \cdot \text{input}$
自表达特征输出

返回 : c(系数矩阵), x2(自表达特征)

独立优化器 : optimizer1 (Adam)
仅更新 SelfExpr + DA 模块参数

MFL (多因子少样本学习):

输入 : x2(自表达特征), C(系数矩阵)

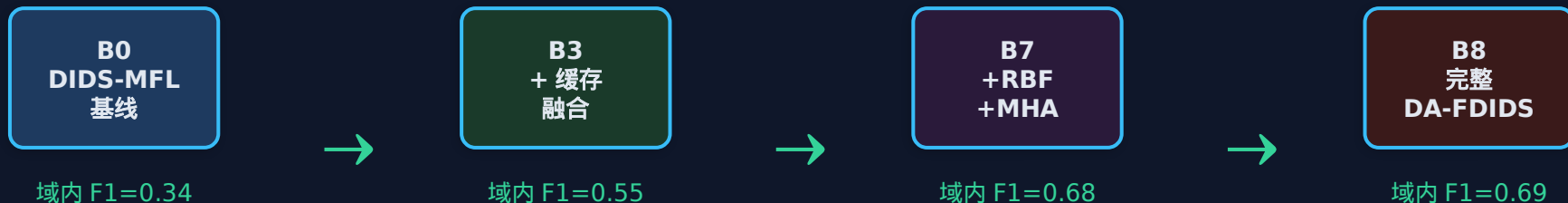
- ① MFL(x2, C, $\gamma=0.1$, $\eta=0.1$, $\alpha=0.1$)
→ ADMM 交替方向乘子法优化
→ 最多 80 次迭代 , 容差 $1e-4$
→ 返回增强相似度矩阵 L1
- ② $C1 = C + \varepsilon \cdot L1$ ($\varepsilon=0.3$)
将 ADMM 增强项叠加到系数矩阵
- ③ enhance_sim_matrix(C1,
way=5, subspace_dim=4, iter=1)
→ SVD 子空间聚类
→ 返回最终相似度矩阵 L

相似度 → 类别分数 → 预测 :

```
mat = L[:num_sup, num_sup:]  
for each class c in [0,1,2,3,4]:  
    mask = (support_labels == c)  
    scores[:,c] = mean(mat[mask, :], dim=0)  
    scores = (scores - min) / (max - min)  
    preds = argmax(scores, dim=1)  
    loss_task = CrossEntropy(scores, true_labels) + ||x2 - concat|| # 交叉熵 + 重构损失
```

提取支撑 × 查询子矩阵 (25×75)
对 5 个攻击类别
该类别的支撑样本掩码
该类支撑样本的平均相似度
min-max 归一化到 [0,1]
最大相似度类别为预测结果

总结：最优组件路径与关键发现



关键发现：

- B0→B3 (缓存融合): 单次最大 F1 跃升 — 域内 +62%，跨域 +137%，漂移 +604%
- B3→B7 (RBF+MHA): 第二次显著跃升 — 域内 +23%，跨域 +31%
- GRL/MMD/ 稳定 LoRA (B4-B6): 域内轻微退化，但跨域各贡献 +6~11% 的提升
- 最优 $\alpha = 0.0 \sim 0.3$: 纯缓存检索在绝大多数场景下优于纯 MFL 相似度学习
- 跨域 F1(0.37) 远低于域内 F1(0.69) — 跨域泛化仍是核心挑战和未来工作方向
- 基础编码器 + LoRA (B1-B2) 单独使用时域内性能反而下降，必须与缓存融合 (B3) 配合
- DA 模块 (GRL/MMD) 对跨域有效，但对域内无益 — 应按场景选择性启用